Remote State, State Mgmt & Code Quality / CI

S3

tflint

checkov

OPA

AWS Academy Learner Lab — Environment & Constraints

WEEK 15

Read before starting Week 15 lab — these rules are enforced by the Lab policy

Regions	us-east-1 or us-west-2 only All resources must be created in one of these two regions.
Credentials	Provided by the Lab panel — click "AWS Details" → copy Access Key/Secret Key. Export as environment variables or use the credential file. NEVER hardcode in .tf files.
EC2 Instance Types	nano, micro, small, medium, large only. t3.micro recommended. xlarge and above are automatically terminated by the Lab monitor.
IAM — CRITICAL	Cannot create new IAM roles. Use pre-existing LabRole and LabInstanceProfile. PassRole is allowed only for specific pre-defined roles.
EBS Volumes	Types: gp2, gp3, sc1, standard only (no io1/io2). Max size: 100 GB. Enforced by policy.
No Spot/Reserved	Spot, Reserved, and Scheduled instances are denied. Only on-demand, default tenancy.

Week 15 — Specific Lab Notes

S3 backend bucket

The SCP denies s3:GetBucketObjectLockConfiguration, which the provider reads on EVERY aws_s3_bucket refresh.
Terraform cannot manage an aws_s3_bucket here at all — create the bucket with the AWS CLI

State locking

Set use_lockfile = true in the backend block — no DynamoDB table required.
Terraform writes a .tflock object next to the state file in S3.

Backend credentials

The S3 backend reads credentials from the same env vars as the provider.
Re-export AWS credentials at the start of each session before terraform init.

Security scanning

Install checkov: pip install checkov (or use Cloud9/CloudShell).
Expected findings: S3 bucket versioning, encryption, public-access block.
Remediate at least one finding — add lifecycle / server_side_encryption_configuration.

Week 15 Agenda

WEEK 15

3 hours: ~60 min concept + ~120 min hands-on lab

- 6.4 — Remote backends — S3, GCS, Azure; backend block; workspaces; migration
- 6.5 — State manipulation — backup/restore, state mv/rm/import, CLI-driven changes
- 6.6 — State drift — accidental/intentional manual changes, conflicting automation
- 6.7 — Cross-project state — terraform_remote_state data source, structuring
- 6.8 — State-only resources — random, time, null providers, terraform_data
- 7.3 — Quality tools — terraform validate, Terratest, testing framework, TFLint
- 7.4 — Security scanning — Checkov, Trivy, Snyk, Checkmarx, Mend
- 7.5 — Custom policy — OPA with TFLint, custom Checkov rules
- **LAB: S3 backend migration, state manipulation, validate + tflint + checkov**

Remote State Backends

Moving state out of local files and into shared, secure storage | Ch. 6.4

AWS S3 Backend (recommended for AWS)

```
# First: create the S3 bucket (no lock table needed)
terraform {
  backend "s3" {
    bucket         = "my-tf-state-bucket"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true     # AES-256 at rest
    use_lockfile   = true     # S3 native locking
  }
}

# Migrate local → remote:
terraform init    # re-run after adding backend
# Terraform prompts: copy local state? → yes
```

Terraform writes a .tflock object beside the state.
dynamodb_table is deprecated (removed in a future release).

Backend Comparison

`local`

Default. Single file on disk. No locking. Dev only.

`s3`

AWS S3 with use_lockfile. Most common for AWS teams.

`azurerm`

Azure Blob Storage + lease-based locking.

`gcs`

Google Cloud Storage. Built-in object locking.

`http`

Generic HTTP backend. DIY state server.

`Terraform Cloud`

cloud {} block. Managed state + RBAC + runs.

Workspaces: terraform workspace new staging — separate state per env within same backend.

Manipulating State

Surgical state operations via CLI | Ch. 6.5

terraform state list

List all resources in state

```
terraform state list
terraform state list
aws_instance.*
```

terraform state show

Show all attributes of one resource

```
terraform state show
aws_instance.web
# Prints full attribute set
```

terraform state mv

Rename/move a resource in state without destroy.
Essential when refactoring module paths.

```
terraform state mv
aws_instance.web \
  module.ec2.aws_instance.web
```

terraform state rm

Remove from state WITHOUT destroying.
Use when handing resource to another config.

```
terraform state rm
aws_instance.old
# Resource stays in AWS, gone
from state
```

terraform import

Bring existing resource into state.
Config must already exist for the resource.

```
terraform import
aws_instance.web i-0abc
# Or block form: import { to=...
id=... }
```

Backup/Restore

State is backed up automatically before each apply.
Restore: terraform state push backup.tfstate

```
# Backup:
.terraform.tfstate.backup
terraform state pull >
manual_backup.json
terraform state push backup.json
```

Golden rule: Always back up state before any manual manipulation. Use CLI — never text-edit the JSON.

State Drift & Cross-Project State

When real world diverges — and how to share state | Ch. 6.6–6.7

State Drift (Ch. 6.6)

Accidental manual change

Someone edited a resource in the console.
Next plan shows unexpected changes.

Intentional manual change

Emergency hotfix applied manually.
Must import or accept drift.

Conflicting automation

Another tool (CloudFormation, script) also manages the resource.

Terraform errors

Apply failed mid-run. State partially updated.

```
Detect drift:
terraform plan -refresh-only
terraform refresh (updates state)
```

terraform_remote_state (Ch. 6.7)

```hcl
# network/outputs.tf
output "vpc_id" {
  value = aws_vpc.main.id
}

# app/main.tf — consume network state
data "terraform_remote_state" "net" {
  backend = "s3"
  config = {
    bucket = "my-tf-state"
    key    = "network/terraform.tfstate"
    region = "us-east-1"
  }
}
resource "aws_instance" "app" {
  subnet_id = data.terraform_remote_state.net.outputs.vpc_id
}
```

- *Alternative: Use SSM Parameter Store to publish outputs*
- *Alternative: Decouple with data sources (aws_vpc)*
- *Tightly coupled states are hard to refactor — use sparingly*

Maintaining Code Quality

Validate, lint, and test Terraform configs | Ch. 7.3

terraform validate

Checks syntax and internal consistency.
Does NOT call the AWS API — runs offline.
Always run before plan.

```
terraform validate
# Success message or list of errors
# Catches: undefined variables, bad refs
```

terraform fmt

Canonical HCL formatting.
-check: exit non-zero if changes needed (CI).
-recursive: format all subdirectories.

```
terraform fmt -check -recursive
terraform fmt -recursive
# Rewrites files in-place
```

TFLint

Linter that goes beyond syntax.
Detects provider-specific issues (invalid instance types, missing required args).
Plugin-based for AWS/Azure/GCP.

```
tflint --init
tflint --recursive
# .tflint.hcl config file
```

Terraform Test Framework

Built-in since Terraform 1.6.
Write .tftest.hcl files with run blocks.
Actually provisions resources — integration test.

```
terraform test
# Reads *.tftest.hcl files
# run { command = plan/apply }
```

Terratest

Go-based testing library.
Provisions real infra, asserts outcomes, destroys.
Powerful but slow — for critical modules.

```
# Go test file
terraform.Init(t, opts)
terraform.Apply(t, opts)
defer terraform.Destroy(t, opts)
```

Pre-commit hooks

Run fmt, validate, tflint on every git commit.
Fast feedback before CI.
pre-commit install in repo root.

```
# .pre-commit-config.yaml
repos:
- repo: antonbabenko/pre-commit-terraform
  hooks: [terraform_fmt, terraform_validate]
```

Validating Security

Static analysis for misconfigurations before deploy | Ch. 7.4

Checkov

- Python-based, 1000+ built-in checks
- Supports Terraform, CF, K8s, ARM, Dockerfile
- `pip install checkov → checkov -d .`
- Output: PASSED / FAILED per resource
- CI-friendly: --soft-fail, --output junitxml

Trivy (was TFSec)

- Go-based, very fast scanner
- `trivy config . for Terraform`
- Checks: open SGs, unencrypted S3, public RDS
- SARIF output for GitHub Advanced Security
- Container scanning too (bonus)

Snyk

- `SaaS + CLI. snyk iac test`
- Prioritizes by severity + fix availability
- Integrates with GitHub/GitLab PRs
- Free tier available for open source

Checkmarx / Mend

- Enterprise tools with SAST + DAST
- Deeper analysis, license scanning
- Expensive; for regulated industries
- `Often part of enterprise CI pipeline`

Common findings: S3 bucket public read, SG allows 0.0.0.0/0 on 22/3389, EBS not encrypted, RDS not encrypted at rest, IAM wildcard (*) actions.

Custom Policy Enforcement

OPA with TFLint and custom Checkov rules | Ch. 7.5

Open Policy Agent (OPA)

Policy-as-code engine. Write rules in Rego language.
Evaluate Terraform plan JSON against policies before apply.

```
# policy/instance_type.rego
package terraform.aws

allowed_types = ["t3.micro","t3.small"]

deny[msg] {
  r := input.resource_changes[_]
  r.type == "aws_instance"
  t := r.change.after.instance_type
  not allowed_types[_] == t
  msg := sprintf("Instance type %v not allowed", [t])
}

# Run:
terraform plan -out=plan.bin
terraform show -json plan.bin > plan.json
opa eval -d policy/ -i plan.json "data.terraform.aws.deny"
```

Custom Checkov Rules

Extend Checkov with Python rules for org-specific requirements.

```
# checks/require_tags.py
from checkov.terraform.checks.resource.base \
  import BaseResourceCheck

class RequiredTagsCheck(BaseResourceCheck):
  id   = "CKV_CUSTOM_1"
  name = "Ensure required tags"
  supported_resources = ["aws_instance"]

  def scan_resource_conf(self, conf):
    tags = conf.get("tags", [{}])[0]
    return "Environment" in tags \
      and "Owner" in tags

scanner = RequiredTagsCheck()

# Run with custom rule directory:
checkov -d . --external-checks-dir checks/
```

HANDS-ON LAB

Week 15 Lab — S3 Backend, State Management & Security Scanning

Objective: Migrate to S3 remote backend, manage state, and run quality/security gates

1

Create S3 backend

Bucket via AWS CLI (SCP limit)
TF manages versioning/encryption
backend "s3" + use_lockfile

2

State locking demo

Run two applies concurrently
Observe lock error message
Find the .tflock object in S3

3

State manipulation

terraform state mv a resource
terraform state rm + import
Backup and restore state

4

Remote state

Consume another project's outputs
via terraform_remote_state
Add random suffix to S3 name

5

Quality scan

terraform validate && tflint
Install & run checkov / trivy
Fix at least one finding

6

Stretch: CI hook

pre-commit hook: fmt+validate
GitHub Actions workflow stub
Security scan in CI step

Deliverable: S3 backend with locking, passing validate/tflint, security scan report with remediated finding

Week 15 Lab — S3 Backend, State Mgmt & Security Scanning (AWS Academy)

Objective: Migrate to S3 remote backend, manage state, run tflint + checkov — all permitted in Learner Lab

1

Create S3 backend

Bucket created with AWS CLI
aws_s3_bucket is blocked by SCP
TF manages its configuration

2

S3 native locking

use_lockfile = true
Lock object: <key>.tflock
No DynamoDB table needed

3

Migrate state

Add backend "s3" block
terraform init → yes to copy
Verify state in S3 console

4

State manipulation

terraform state mv / rm
terraform import a resource
Backup: state pull > backup.json

5

Quality scan

terraform validate && tflint
pip install checkov
checkov -d . → fix ≥1 finding

6

Stretch: remote state

Consume another project via
terraform_remote_state
Add random_string suffix to bucket

Week 15 — Key Takeaways & Course Summary

WEEK 15

You now have a production-ready Terraform workflow

Remote Backend

S3 with use_lockfile for team-safe state. Encrypted at rest. Locking prevents concurrent corruption.

State Manipulation

state mv / rm / import for refactoring. Always backup first. Never hand-edit the JSON.

State Drift

terraform plan -refresh-only detects drift. Manual changes need import or accept. Conflicting tools = danger.

Quality Gates

validate + fmt + tflint = minimum CI bar. Run on every commit via pre-commit hooks.

Security Scanning

Checkov or Trivy catches misconfigs before deploy. Integrate in CI so findings block merge.

Custom Policy

OPA or custom Checkov rules for org standards. Policy-as-code = enforceable, version-controlled.

Course complete · You can now provision AWS infrastructure with HCL, modules, expressions, remote state, and automated quality + security gates ✓